

category: networking-protocol

tags: [socket, tcp, cancellation, wsacancelblockingcall, accept, thread-pool]

severity: medium

created: 2026-07-21

updated: 2026-07-21

project-origin: iPGSv4 (IPGS.Kiosk.Avalonia)

⚠ **CẬP NHẬT QUAN TRỌNG:** Bản giải pháp đầu tiên (`catch (SocketException) when (ct.IsCancellationRequested)` đặt NGOÀI `Task.Run`, ở tầng `AcceptLoop`) KHÔNG đủ — VS debugger vẫn break "User-Unhandled" y hệt. Xem mục "Giải pháp — bản ĐÚNG" bên dưới: phải bắt exception NGAY TRONG lambda đồng bộ của `Task.Run`, không được để nó propagate qua `await` rồi mới catch ở ngoài.

Socket.Accept() blocking + Close() để dừng → SocketException (WSACancelBlockingCall) không phải OperationCanceledException

Tình huống gặp phải

`SocketServerImpl.AcceptLoop()` chạy `_listener.Accept()` (đồng bộ, blocking) bên trong `Task.Run(..., ct)` để có thể chờ bất đồng bộ. Khi `StopAsync()` được gọi: `_cts.Cancel()` rồi `_listener.Close()` để đánh thức thread đang block ở `Accept()`.

Triệu chứng / Lỗi

Visual Studio debugger break với dialog "Exception User-Unhandled":

```
System.Net.Sockets.SocketException: 'A blocking operation was interrupted by a call to WSACancelBlockingCall.'
```

Xảy ra tại dòng `var client = await Task.Run(() => _listener!.Accept(), ct).`

Nguyên nhân gốc rễ (Root Cause)

`CancellationToken` truyền vào `Task.Run` chỉ hủy được nếu task **chưa bắt đầu chạy** — không thể hủy giữa chừng một cuộc gọi đồng bộ blocking như `Socket.Accept()`. Cách duy nhất để đánh thức thread đang block trong `Accept()` là đóng socket (`Close()/Dispose()`), và trên Windows việc đó khiến Winsock ném `SocketException` với message "WSACancelBlockingCall" — **không phải** `OperationCanceledException`. Code cũ chỉ có `catch (OperationCanceledException) { break; }` rồi rơi xuống `catch (Exception)` chung, khiến VS coi đây là lỗi thật và (do có bật "Break when this exception type is user-unhandled") dừng debugger giữa luồng tắt server hoàn toàn bình thường.

Lớp nguyên nhân thứ 2 (phát hiện sau khi áp bản fix đầu vẫn không hết break): Ngay cả khi thêm `catch (SocketException) when (ct.IsCancellationRequested)` ở tầng ngoài (trong `AcceptLoop`, bao

quanh `await Task.Run(...)`, VS debugger **vẫn** break "User-Unhandled". Lý do: exception được ném ra bên trong lambda đồng bộ chạy trên thread pool (`Task.Run`), rồi mới propagate lên qua điểm `await` ở stack frame khác. VS's "break when user-unhandled" heuristic cần đánh giá exception filter (`when`) để biết catch có thật sự bắt được không — nhưng việc đánh giá đó xuyên qua ranh giới `Task.Run` → `await` không được phân tích tin cậy, nên debugger vẫn báo break dù runtime thực tế sẽ catch đúng. Đây là hạn chế đã biết của VS debugger với `async/Task.Run`, không phải lỗi logic code.

Giải pháp — bản ĐÚNG (bắt exception ngay trong lambda đồng bộ, cùng stack frame với throw)

```
var client = await Task.Run(() =>
{
    try { return _listener?.Accept(); }
    catch (SocketException) when (ct.IsCancellationRequested)
    {
        // Listener bị Close() từ StopAsync() trong lúc Accept() đang block
        // (WSACancelBlockingCall trên Windows) — luồng dừng server bình thường.
        return null;
    }
    catch (ObjectDisposedException)
    {
        return null;
    }
}, ct).ConfigureAwait(false);

if (client is null)
{
    if (ct.IsCancellationRequested) break;
    continue;
}

_ = Task.Run(() => HandleClient(client), CancellationToken.None);
```

1. Bắt exception **ngay bên trong lambda `Task.Run`** — cùng stack frame đồng bộ với nơi `Accept()` ném ra — thay vì để nó propagate qua `await` rồi mới catch ở tầng `AcceptLoop`. Đây là điểm khác biệt mấu chốt so với bản fix đầu (catch ngoài, dùng `when`) — bản đó VẪN bị VS break nhầm.
2. Trả về `null` khi bị cancel/dispose thay vì rethrow; ở tầng gọi kiểm tra `client is null` để quyết định `break` hay `continue`.
3. Giữ `catch (OperationCanceledException) { break; }` và `catch (Exception) (lỗi thật)` ở tầng `AcceptLoop` như cũ.
4. Bọc `Task.Delay(500, ct)` trong catch chung bằng try/catch `OperationCanceledException` — nếu `ct` bị cancel ngay trong lúc delay thì thoát êm thay vì để exception thoát ra ngoài `AcceptLoop`.

Áp dụng lại (How to reuse)

- Bất kỳ chỗ nào dùng `Socket.Accept()/Receive()/Send()` đồng bộ chạy trong `Task.Run` kèm hủy bằng cách đóng socket → PHẢI bắt exception **ngay trong lambda `Task.Run`** (không phải ở tầng `catch` bao quanh `await` bên ngoài), rồi trả `null`/giá trị sentinel để tầng gọi tự quyết định tiếp tục hay dừng.

- Nếu thấy debugger vẫn break "User-Unhandled" dù đã thêm `catch...when` ở tầng ngoài bao quanh `await Task.Run(...)` → đừng cố thêm điều kiện `when` phức tạp hơn, chuyển hẳn catch vào trong lambda đồng bộ.
- Nguyên tắc chung: **catch càng gần nơi ném ra (cùng stack frame đồng bộ) càng đáng tin cậy với công cụ debug/phân tích tĩnh** — đặc biệt khi exception phải "băng qua" ranh giới `Task.Run/await/thread pool`.

Chú ý / Cạm bẫy (Gotchas)

- ⚠ Đừng bắt `SocketException` một cách chung chung (không kèm điều kiện `ct.IsCancellationRequested`) — sẽ nuốt luôn lỗi mạng thật xảy ra khi server đang chạy bình thường (client bị rớt mạng giữa chừng khi đang `Accept` là rất hiếm nhưng vẫn cần phân biệt).
- ⚠ `SocketException` do `WSACancelBlockingCall` chỉ tái hiện trên Windows (Winsock) — trên Linux/macOS việc đóng socket khi đang `Accept()` có thể ném `ObjectDisposedException` hoặc `SocketException` khác (ví dụ `Interrupted`), nên vẫn cần catch dựa trên `ct.IsCancellationRequested` thay vì chỉ dựa vào message.
- ⚠ Đừng tin `catch...when` đặt ở tầng ngoài bao quanh `await Task.Run(...)` sẽ ngăn VS debugger break — dù runtime xử lý đúng 100%, hành vi debug vẫn gây hiểu nhầm là "còn lỗi". Luôn verify bằng cách chạy lại (F5) sau khi sửa, không chỉ đọc code.

Tham chiếu

- Project liên quan: iPGSv4 — `IPGS.Kiosk.Avalonia/Services/Implementations/SocketServerImpl.cs`